



RAPPORT DE SÉCURITÉ

Projet MiniNotes — Option B

Formation OWASP & Secure Coding

Sandrine MERITEL

Juin 2026

<https://github.com/Adn3559/mininotes>

Table des matières

I.	Introduction	5
II.	Préparer le terrain	5
A.	La route des notes (lister / créer) (VULNÉRABLE)	5
B.	La route « une note par id » (IDOR).....	6
III.	Vérifier que MiniNotes tourne	7
A.	Lancer l'app.....	7
IV.	Confirmer qu'elle est bien trouée (ta preuve « AVANT »).....	8
A.	Login normal (pour comparer)	8
B.	Injection SQL — bypass du login (sans mot de passe)	8
C.	IDOR — lire la note privée de l'admin.....	9
V.	Versionner l'état initial	10
VI.	L'audit : trouver et nommer (avant de toucher au code).....	11
VII.	Les correctifs (cause racine, faille par faille).....	16
A.	Faille A — Injection SQL sur le login (app/api/login).....	16
B.	Faille B — Mots de passe en clair (lib/sqlldb.ts + app/api/login)	18
C.	Faille C — Fuite de données + message bavard (app/api/login)	19
D.	Faille D — IDOR sur app/api/notes/[id]	20
E.	Faille E — XSS stocké (app/commentaires/page.tsx)	21
F.	Faille F — Cookie de session volable (app/api/login).....	23
G.	Faille G — CSRF sur app/api/profil	24
H.	Faille H — Pas de validation d'entrée (app/api/notes POST)	25
I.	Faille I — Brute force (app/api/login).....	27
VIII.	Durcissement & secrets	29
A.	Faille J — Secret en dur (lib/config.ts)	29
B.	Faille K — En-têtes de sécurité absents.....	30
C.	Faille L — Dépendances & Dependabot	32
IX.	La CI/CD qui bloque le merge	33
X.	Prouver (les attaques doivent ÉCHOUER).....	36
XI.	Limites & Points d'amélioration.....	37
XII.	Conclusion	38

Table des illustrations

Figure 1 : Etat initiale vulnérable	5
Figure 2 : IDOR.....	6
Figure 3 : npm run dev.....	7
Figure 4 : page d'accueil de MiniNotes	7
Figure 5 : Page commentaires	7
Figure 6 : login normal.....	8
Figure 7 : injection SQL.....	8
Figure 8 : IDOR.....	9
Figure 9 : GtiHub.....	10
Figure 10 : installation ESLint.....	11
Figure 11 : premier ESLint qui plante	12
Figure 12 : eslint.config.mjs final	12
Figure 13 : ESLint passe à 0.....	13
Figure 14 : npm audit -- audit-level=high	13
Figure 15 : Semgrep 0 findings	14
Figure 16 : Preuve avant - fuite de données.....	16
Figure 17 : Injection bloquée	16
Figure 18 : log serveur après	16
Figure 19 : Non-régression	17
Figure 20 : hash bcrypt visible	18
Figure 21 : log serveur après	18
Figure 22 : Preuve après	19
Figure 23 : log serveur	20
Figure 24 : Preuve après	20
Figure 25 : Log du serveur	21
Figure 26 : Preuve avant	22
Figure 27 : Preuve après	22
Figure 28 : Preuve après	23
Figure 29 : attaque bloquée	24
Figure 30 : récupération du jeton	24
Figure 31 : usage légitime.....	24
Figure 32 : log serveur.....	25
Figure 33 : non-régression complète.....	26
Figure 34 : log serveur	26
Figure 35 : Vérification du .gitignore	27
Figure 36 : Triple confirmation	27
Figure 37 : Preuve après	29
Figure 38 : log du serveur.....	29
Figure 39 : brut force bloquée	31
Figure 40 : log serveur	31
Figure 41 : Preuve après	32
Figure 42 : Tests locaux avant push	34
Figure 43 : CI verte sur main.....	34
Figure 44 : Démo de blocage sur GitHub.....	35
Figure 45 : Résultat des checks sur la PR de démo.....	35

Figure 46 : Les trois attaques rejouées 36

Figure 47 : log serveur 36

I. Introduction

Dans le cadre de cette formation OWASP & Secure Coding, j'ai choisi l'Option B — MiniNotes, une application de prise de notes fictive fournie sous forme de code troué, jamais vue avant l'audit. Ce choix correspond à un scénario réaliste : arriver sur une codebase inconnue et en évaluer la sécurité sans en connaître l'historique.

Stack technique : Next.js 16 + TypeScript, moteur SQL en mémoire alasql, bcryptjs pour le hachage, Zod pour la validation. L'application expose 5 routes API (/api/login, /api/notes, /api/notes/[id], /api/profil) et une page publique /commentaires.

II. Préparer le terrain

Le projet a été généré avec create-next-app, le moteur alasql ajouté comme dépendance, et tous les fichiers de routes créés volontairement vulnérables pour simuler une application réelle avec des failles. Le tout versionné sur GitHub dès le premier commit (projet: etat initial vulnerable) afin de garder une trace claire de l'état AVANT correctifs.

A. La route des notes (lister / créer) (VULNÉRABLE)

Lors de la mise en place du fichier route.ts pour notes on voit un message rouge apparaitre.

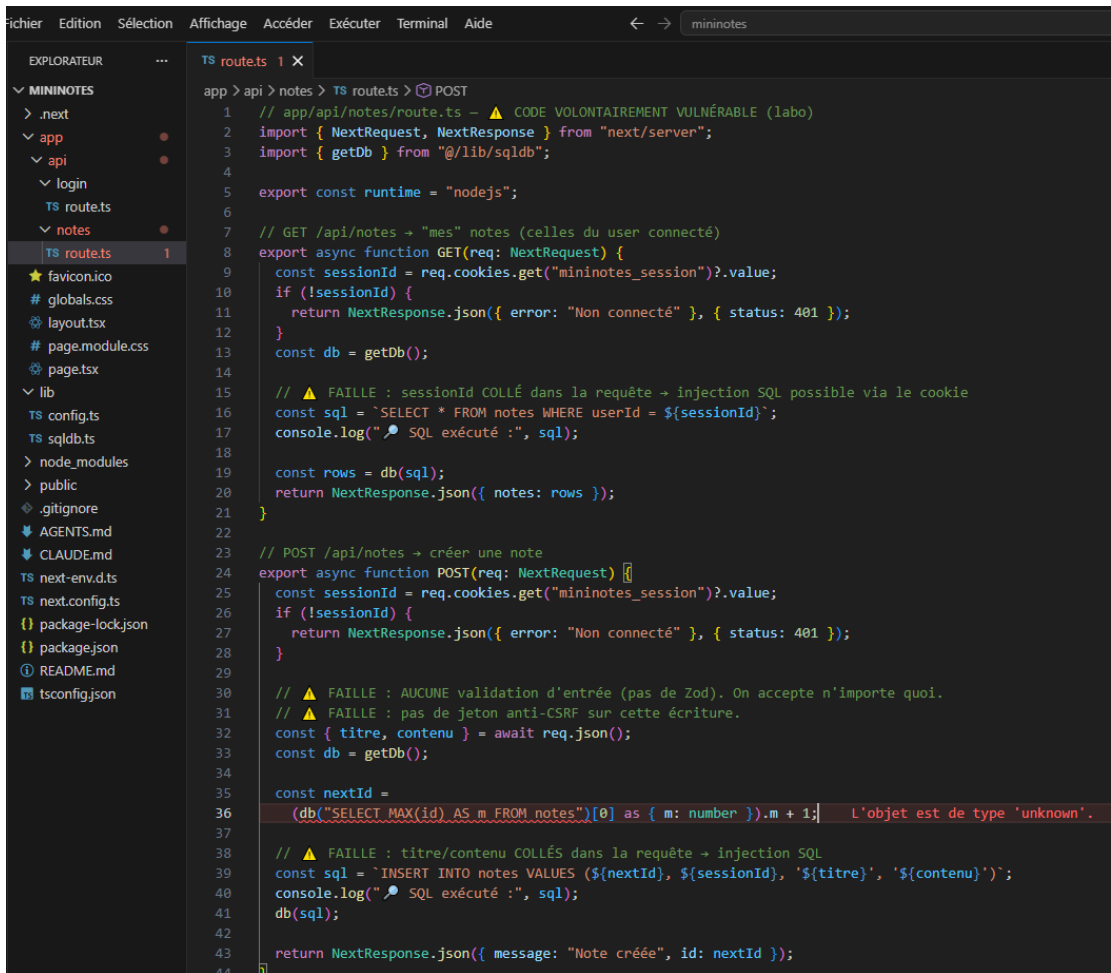


Figure 1 : Etat initiale vulnérable

C'est normal et c'est exactement le message attendu dans ce contexte : `db("SELECT MAX(id) AS m FROM notes")[0]`, `alasql` n'a pas de types TypeScript définis, donc TypeScript ne sait pas deviner précisément ce que `db(...)` renvoie (il le voit comme `any` ou un type générique), et le `as { m: number }` rentre alors en conflit avec son inférence. C'est du typage, pas une erreur de logique : le code fonctionnera très bien à l'exécution.

Pour l'instant, ce n'est pas bloquant : on continue, et le serveur dev (qui utilise généralement une transpilation plus permissive que `tsc --strict`) devrait quand même démarrer.

B. La route « une note par id » (IDOR)

Lors de la mise en place du fichier `route.ts` dans `note [id]`, nous avons eu un autre message en rouge.

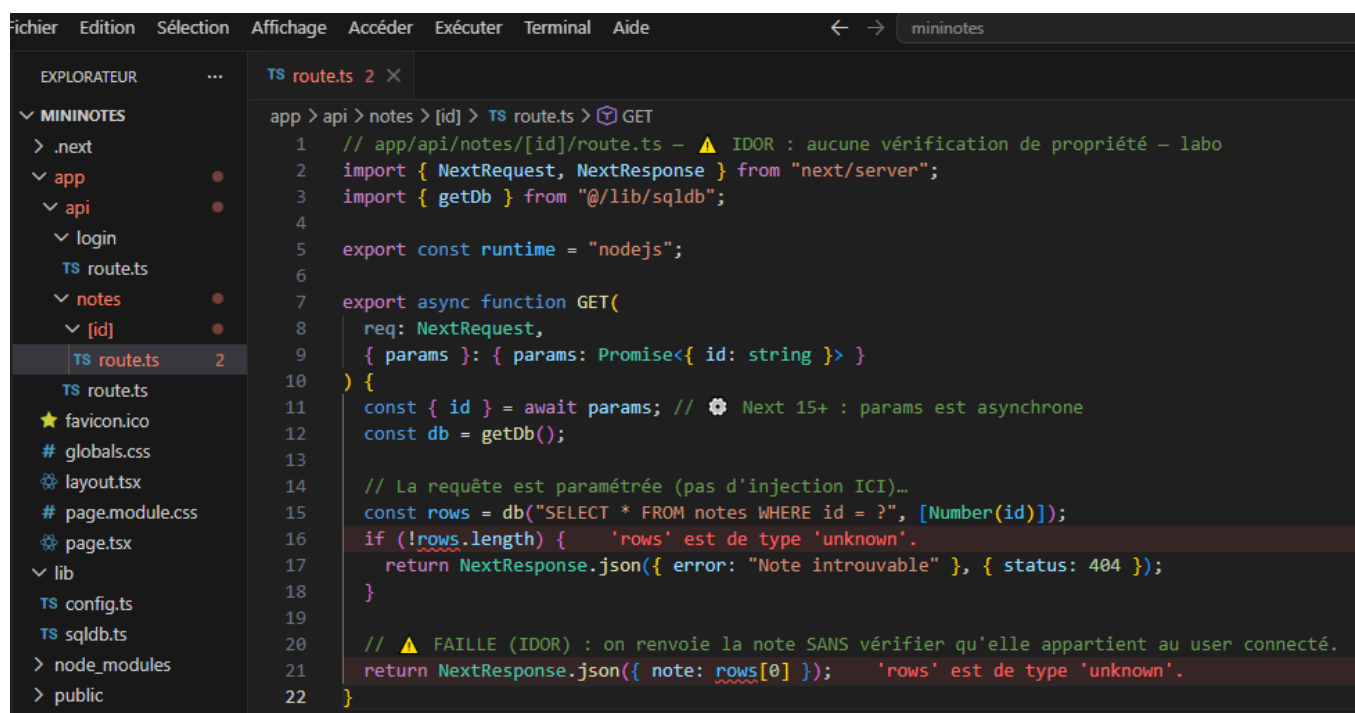


Figure 2 : IDOR

Les 2 messages rouges sont exactement le même phénomène que pour le fichier précédant : `alasql` n'a pas de types TypeScript définis, donc VS Code/TypeScript ne peut pas garantir le type exact de ce que `db(...)` renvoie (un tableau `any[]`), et ça déclenche un avertissement sur `rows.length` et `rows[0]` qui sont en fait corrects à l'exécution. C'est purement cosmétique pour ce labo, pas bloquant, le serveur dev tournera très bien.

III. Vérifier que MiniNotes tourne

A. Lancer l'app

Une fois l'ensemble des fichiers créés — routes API (login, notes, notes/[id], profil), page de commentaires, base de données en mémoire et configuration Next.js — le serveur de développement a été lancé pour confirmer que l'application démarre correctement et que les pages s'affichent bien avant de passer à la phase d'audit.

```
sandrine@DESKTOP-LOGGKCP: ~/secure-coding/mininotes
sandrine@DESKTOP-LOGGKCP:~$ cd ~/secure-coding
sandrine@DESKTOP-LOGGKCP:~/secure-coding$ cd mininotes
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ npm run dev

> mininotes@0.1.0 dev
> next dev

▲ Next.js 16.2.9 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://172.25.114.175:3000
🟢 Ready in 673ms
```

Figure 3 : npm run dev

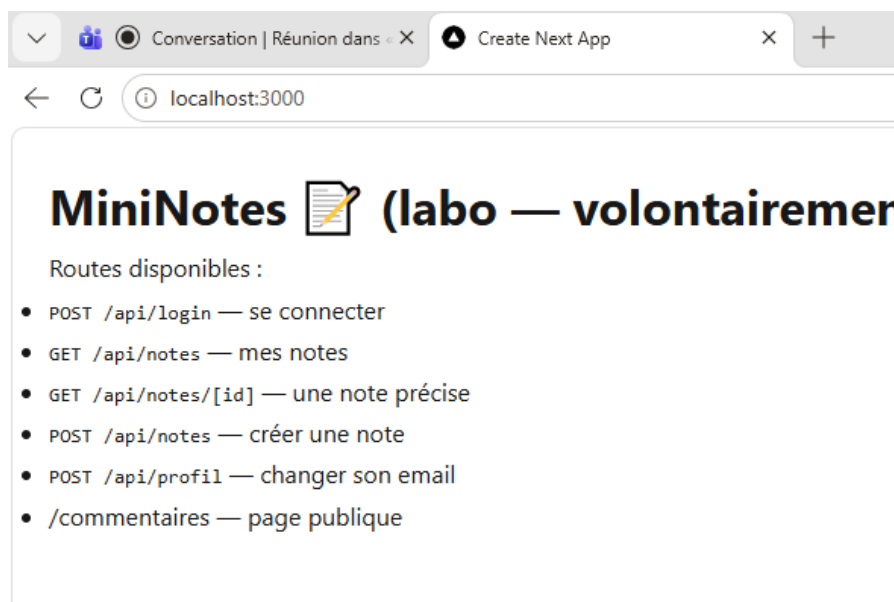
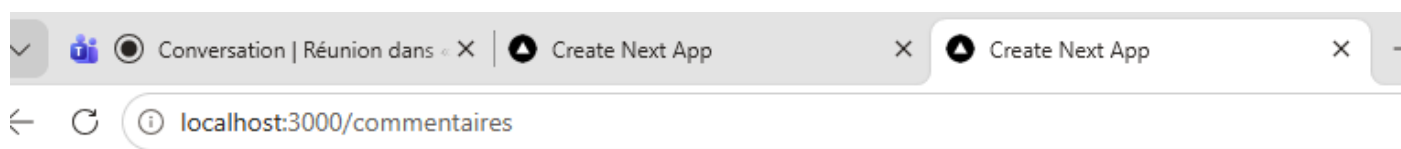


Figure 4 : page d'accueil de MiniNotes



Commentaires

Alice : Super appli !

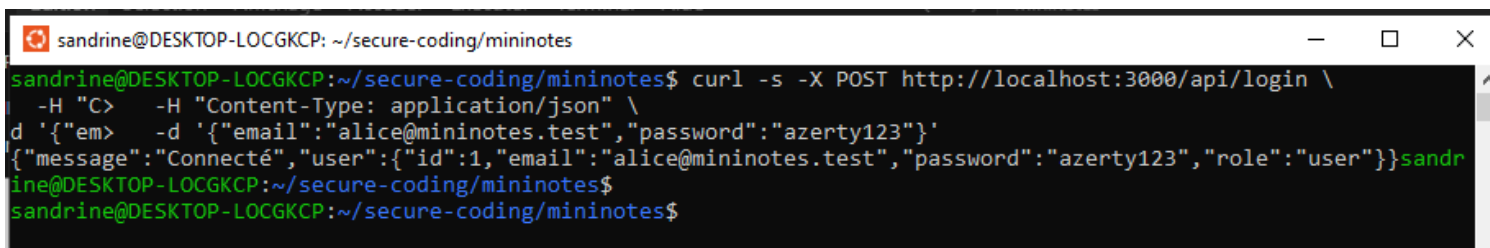
Figure 5 : Page commentaires

IV. Confirmer qu'elle est bien trouée (ta preuve « AVANT »)

Avant de toucher au code, il est indispensable de prouver que les failles existent réellement en les exploitant. Ces trois attaques constituent nos preuves "AVANT" : elles serviront de référence pour confirmer, après chaque correctif, que l'attaque échoue bien (et que l'usage normal fonctionne toujours).

A. Login normal (pour comparer)

Un login avec des identifiants valides devrait renvoyer uniquement les informations strictement nécessaires au client. Or ici, la réponse de l'API expose l'intégralité de l'objet utilisateur, y compris le mot de passe en clair et le rôle. Un attaquant interceptant cette réponse obtient des informations sensibles sans même avoir besoin d'exploiter une faille avancée.

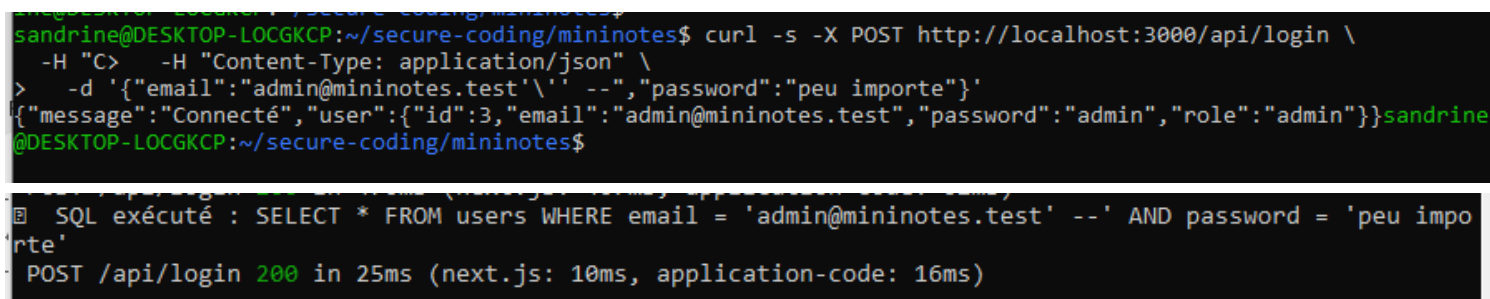


```
sandrine@DESKTOP-LOGCKCP: ~/secure-coding/mininotes
sandrine@DESKTOP-LOGCKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \
-H "C> -H "Content-Type: application/json" \
-d '{"email":"alice@mininotes.test","password":"azerty123"}'
{"message":"Connecté","user":{"id":1,"email":"alice@mininotes.test","password":"azerty123","role":"user"}}sandr
ine@DESKTOP-LOGCKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGCKCP:~/secure-coding/mininotes$
```

Figure 6 : login normal

B. Injection SQL — bypass du login (sans mot de passe)

L'injection SQL est l'une des failles les plus critiques (OWASP A03 — score CVSS 9-10). Elle exploite le fait que la requête SQL est construite par concaténation directe des entrées utilisateur. En insérant le payload `admin@mininotes.test' --` dans le champ email, le `--` transforme la vérification du mot de passe en commentaire SQL, la base ne la voit plus. Résultat : connexion en tant qu'administrateur sans connaître le moindre mot de passe.



```
sandrine@DESKTOP-LOGCKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \
-H "C> -H "Content-Type: application/json" \
-d '{"email":"admin@mininotes.test' --","password":"peu importe"}'
{"message":"Connecté","user":{"id":3,"email":"admin@mininotes.test","password":"admin","role":"admin"}}sandr
ine@DESKTOP-LOGCKCP:~/secure-coding/mininotes$

SQL exécuté : SELECT * FROM users WHERE email = 'admin@mininotes.test' -- AND password = 'peu impo
rte'
POST /api/login 200 in 25ms (next.js: 10ms, application-code: 16ms)
```

Figure 7 : injection SQL

C. IDOR — lire la note privée de l'admin

L'IDOR (Insecure Direct Object Reference, OWASP A01) est un défaut de contrôle d'accès : l'application ne vérifie pas que la ressource demandée appartient bien à l'utilisateur connecté. Ici, Alice (id=1) peut accéder directement à la note id=3 qui appartient à l'administrateur, simplement en modifiant le numéro dans l'URL. Elle récupère ainsi le contenu sensible "le code du coffre est 4271" — sans aucune élévation de privilège technique, juste en devinant un identifiant.

```
{ message : 'connecte', user : { id : 3, email : 'admin@mininotes.test', password : 'admin', role : 'admin' } }
@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -c moi.txt -X POST http://localhost:3000/api/login \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@mininotes.test","password":"azerty123"}' > /dev/null
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b moi.txt http://localhost:3000/api/notes/3
{"note":{"id":3,"userId":3,"titre":"Codes admin","contenu":"le code du coffre est 4271"}}
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 8 : IDOR

Les 3 attaques fonctionnent, votre labo MiniNotes est confirmé réellement vulnérable. On a maintenant toutes les preuves "AVANT" nécessaires :

1. Fuite de données (mot de passe + rôle dans la réponse de login)
2. Injection SQL (bypass d'authentification en admin sans mot de passe)
3. IDOR (lecture de la note privée de l'admin par Alice)

V. Versionner l'état initial

Avant de commencer les correctifs, il est essentiel de capturer l'état vulnérable dans Git. Ce premier commit constitue notre point de référence "AVANT" : il permet au formateur de consulter l'historique et de voir exactement ce qui existait avant toute intervention, et garantit que chaque correctif ultérieur sera traçable et atomique.

On peut voir ici les 7 fichiers créés volontairement vulnérables qui constituent le cœur de MiniNotes : les routes API (login, notes, notes/[id], profil), la page commentaires, la base de données en mémoire (sqldb.ts) et le fichier de configuration avec le secret en dur (config.ts).

Le dépôt est ensuite poussé sur GitHub à l'adresse <https://github.com/Adn3559/mininotes>, rendant le projet visible et consultable par le formateur.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ git init
Reinitialized existing Git repository in /home/sandrine/secure-coding/mininotes/.git/
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ printf "node_modules/\n.next/\n.env.local\nmoi.txt\n" >> .gitignore
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ git add -A
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ git commit -m "projet: etat initial vulnerable (MiniNotes)"
[main 1fea686] projet: etat initial vulnerable (MiniNotes)
12 files changed, 3528 insertions(+), 124 deletions(-)
create mode 100644 app/api/login/route.ts
create mode 100644 app/api/notes/[id]/route.ts
create mode 100644 app/api/notes/route.ts
create mode 100644 app/api/profil/route.ts
create mode 100644 app/commentaires/page.tsx
create mode 100644 lib/config.ts
create mode 100644 lib/sqldb.ts
```

Figure 9 : GtiHub

Récapitulatif de ce qu'on a accompli pour la mise en place :

- MiniNotes installé et fonctionnel (7 fichiers vulnérables créés)
- 3 attaques confirmées avec preuves (fuite de données, injection SQL, IDOR) — à garder précieusement pour le rapport
- Dépôt GitHub créé et premier commit pousser

VI. L'audit : trouver et nommer (avant de toucher au code)

L'audit est une étape fondamentale qui précède tout correctif. L'objectif est d'identifier et de documenter l'ensemble des failles présentes dans le code sans toucher à quoi que ce soit, afin de disposer d'une vision complète avant d'intervenir.

Trois familles d'outils complémentaires ont été utilisées, chacune ayant un angle d'analyse différent :

1. ESLint (SAST léger)

Analyse statique du code source à la recherche de patterns dangereux connus. On lui ajoute des plugins spécialisés : `eslint-plugin-security` (patterns de sécurité génériques), `eslint-plugin-react` (dont la règle `react/no-danger` qui signale `dangerouslySetInnerHTML`) et `eslint-config-next` (règles spécifiques à Next.js). ESLint ne fait pas de suivi de flux de données — il ne relie pas une entrée utilisateur à une requête SQL.

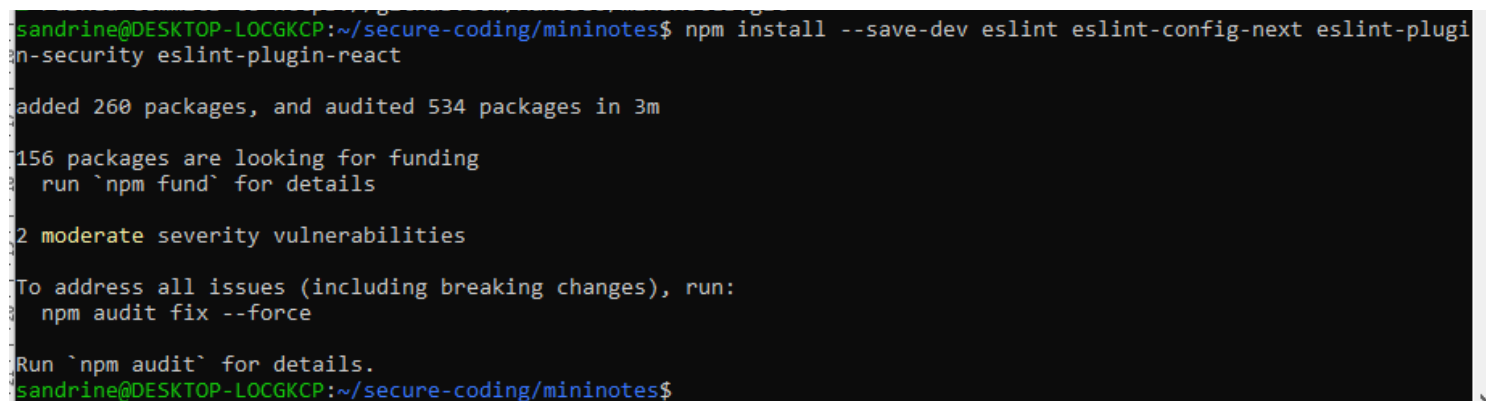
2. npm audit (SCA — Software Composition Analysis)

Analyse les dépendances du projet en les comparant à une base de données de vulnérabilités connues (CVE). Il ne lit jamais notre propre code — il ne peut donc structurellement pas détecter une injection SQL dans nos routes. On utilise l'option `--audit-level=high` pour ne bloquer la CI que sur les vulnérabilités de sévérité haute ou critique.

3. Sengrep (SAST avancé)

Outil de SAST capable de suivi de flux de données (taint analysis) : il relie une entrée utilisateur (source) à un endroit dangereux (sink) en suivant le chemin dans le code. En mode `--config auto`, il utilise le ruleset communautaire de 213 règles. C'est l'outil le plus puissant des trois — mais son angle mort ici est qu'alasql, étant une bibliothèque de niche, n'est pas reconnue comme sink SQL par les règles génériques.

On note le message '2 moderate severity vulnerabilities', ces vulnérabilités modérées dans la dépendance `postcss/next` seront analysées dans la partie npm audit



```
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ npm install --save-dev eslint eslint-config-next eslint-plugin-security eslint-plugin-react
added 260 packages, and audited 534 packages in 3m

156 packages are looking for funding
  run `npm fund` for details

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
```

Figure 10 : installation ESLint

Premier essai d'ESLint — la configuration initiale générait des erreurs car ESLint analysait le dossier `.next` (code compilé automatiquement par Next.js) et rencontrait un bug de compatibilité entre `eslint-config-next` et la version ESLint 9.39.x (erreur `FlatCompat/structure circulaire`). La configuration a dû être ajustée pour ignorer `.next` et utiliser les exports flat natifs des plugins.

```
sandrine@DESKTOP-LOCCKCP:~/secure-coding/mininotes$ npx eslint .
(node:11911) ESLintEmptyConfigWarning: Running ESLint with an empty config (from /home/sandrine/secure-coding/mininotes/eslint.config.mjs). Please double-check that this is what you want. If you want to run ESLint with an empty config, export [{}] to remove this warning.
(Use `node --trace-warnings ...` to show where the warning was created)

/home/sandrine/secure-coding/mininotes/.next/dev/server/chunks/[root-of-the-server] 1o-hm7m. .js
8455:81 warning Unused eslint-disable directive (no problems were reported from 'eqeqeq')
8458:55 warning Unused eslint-disable directive (no problems were reported from 'eqeqeq')
8593:29 warning Unused eslint-disable directive (no problems were reported from 'no-fallthrough')
8875:179 warning Unused eslint-disable directive (no problems were reported from 'no-restricted-syntax')

/home/sandrine/secure-coding/mininotes/.next/dev/server/chunks/[root-of-the-server] 1tlewim. .js
8455:81 warning Unused eslint-disable directive (no problems were reported from 'eqeqeq')
8458:55 warning Unused eslint-disable directive (no problems were reported from 'eqeqeq')
8593:29 warning Unused eslint-disable directive (no problems were reported from 'no-fallthrough')
8875:179 warning Unused eslint-disable directive (no problems were reported from 'no-restricted-syntax')

/home/sandrine/secure-coding/mininotes/.next/dev/server/chunks/[turbopack] runtime.js
23:5 error Definition for rule '@typescript-eslint/no-unused-vars' was not found @typescript-eslint/no-unused-vars
635:1 error Definition for rule '@typescript-eslint/no-unused-vars' was not found @typescript-eslint/no-unused-vars
656:1 error Definition for rule '@typescript-eslint/no-unused-vars' was not found @typescript-eslint/no-unused-vars
677:1 error Definition for rule '@typescript-eslint/no-unused-vars' was not found @typescript-eslint/no-unused-vars
864:5 error Definition for rule '@typescript-eslint/no-unused-vars' was not found @typescript-eslint/no-unused-vars
1247:1 error Definition for rule '@typescript-eslint/no-unused-vars' was not found @typescript-eslint/no-unused-vars
1518:1 error Definition for rule '@typescript-eslint/no-unused-vars' was not found @typescript-eslint/no-unused-vars
1604:1 error Definition for rule '@typescript-eslint/no-unused-vars' was not found @typescript-eslint/no-unused-vars
1662:13 warning Unused eslint-disable directive (no problems were reported from 'no-eval')
```

Figure 11 : premier ESLint qui plante

Configuration finale d'ESLint après résolution du bug de compatibilité — les dossiers `.next` et `node_modules` sont exclus, et les plugins sont importés via leurs exports flat natifs (`nextVitals`, `nextTs`, `security.configs.recommended`) pour contourner le problème `FlatCompat` avec ESLint 9.39.x.

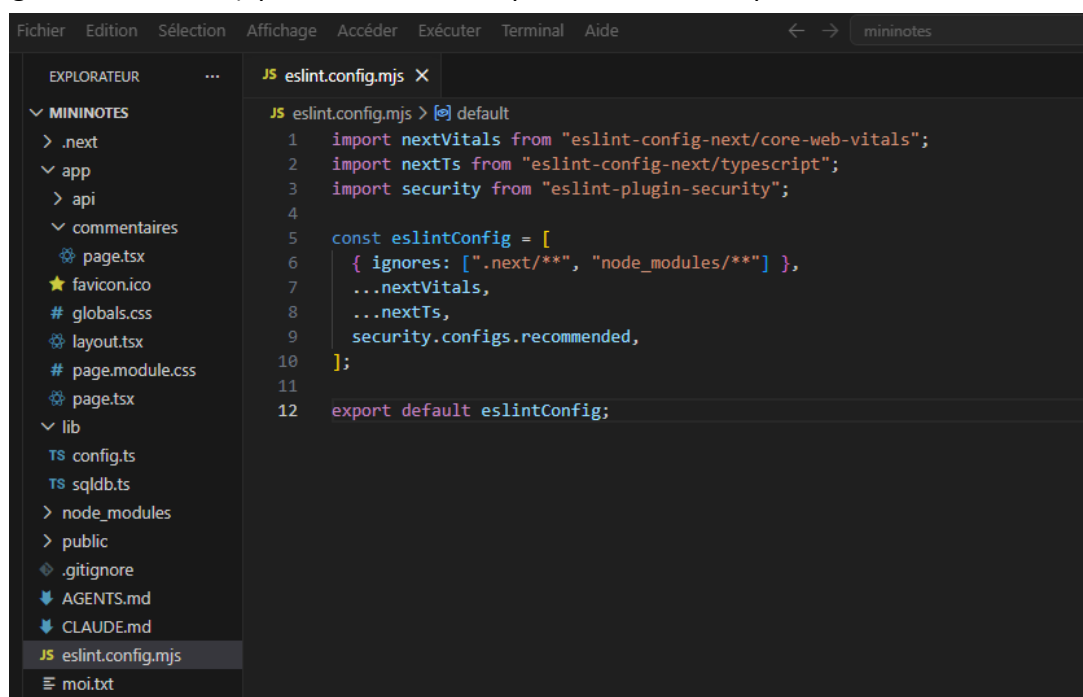


Figure 12 : eslint.config.mjs final

Résultat final d'ESLint : 0 problème détecté, code de sortie 0. Malgré une configuration complète avec les plugins security, react et next, ESLint ne détecte aucune des failles présentes dans le code, ni l'injection SQL par template literal, ni le XSS via dangerouslySetInnerHTML. C'est un faux négatif : 0 alerte ne signifie pas 0 faille

```
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ npx eslint .
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ echo "Code de sortie : $?"
Code de sortie : 0
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
```

Figure 13 : ESLint passe à 0

npm audit --audit-level=high : 2 vulnérabilités de sévérité modérée détectées dans postcss (dépendance indirecte de Next.js). Le correctif suggéré par npm (audit fix --force) impliquerait un downgrade de Next.js 16 vers la version 9.3, un breaking change totalement disproportionné pour un risque modéré en environnement de labo local. Décision : ne pas appliquer ce correctif, le documenter dans les limites. Code de sortie 0 avec --audit-level=high : la CI considère ce résultat comme propre (pas de vulnérabilités high/critical).

```
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ npm audit --audit-level=high
# npm audit report

postcss <8.5.10
Severity: moderate
PostCSS has XSS via Unescaped </style> in its CSS Stringify Output - https://github.com/advisories/GHSA-qx2v-qp2m-jg93
fix available via `npm audit fix --force`
Will install next@9.3.3, which is a breaking change
node_modules/postcss
  next 9.3.4-canary.0 - 16.3.0-canary.5
  Depends on vulnerable versions of postcss
  node_modules/next

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ echo "Code de sortie : $?"
Code de sortie : 0
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
```

Figure 14 : npm audit -- audit-level=high

Semgrep en mode --config auto : 213 règles exécutées sur 26 fichiers, 0 finding. Résultat identique à ESLint — Semgrep ne détecte pas l'injection SQL car alasql est une bibliothèque de niche absente du ruleset communautaire : aucune règle ne reconnaît db(`...\${variable}...`) comme un sink SQL dangereux. C'est un faux négatif classique, et c'est exactement pour ça qu'une règle Semgrep maison (regles/sqli.yml) sera créée plus tard dans le pipeline CI

```
sandrine@DESKTOP-LOCGKCP:~/secure-coding/mininotes$ semgrep --version
1.166.0
sandrine@DESKTOP-LOCGKCP:~/secure-coding/mininotes$ semgrep scan --config auto .
```

ooo

Semgrep CLI

```

[+] Loading rules from registry...
Scanning 26 files (only git-tracked) with:

[+] Semgrep OSS
    [+] Basic security coverage for first-party code vulnerabilities.

[+] Semgrep Code (SAST)
    [+] Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

[+] Semgrep Supply Chain (SCA)
    [+] Find and fix the reachable vulnerabilities in your OSS dependencies.

[+] Get started with all Semgrep products via `semgrep login`.
[+] Learn more at https://sg.run/cloud.

100% 0:00:00
```

Scan Summary

```

[+] Scan completed successfully.
  • Findings: 0 (0 blocking)
  • Rules run: 213
  • Targets scanned: 26
  • Parsed lines: ~100.0%
  • Scan was limited to files tracked by git
  • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 213 rules on 26 files: 0 findings.
[+] Missed out on 1859 pro rules since you aren't logged in!
[+] Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)

[+] If Semgrep missed a finding, please send us feedback to let us know!
    See https://semgrep.dev/docs/reporting-false-negatives/
sandrine@DESKTOP-LOCGKCP:~/secure-coding/mininotes$
```

Figure 15 : Semgrep 0 findings

Q0.1 — Quel outil voit l'injection SQL du login ? Lequel la rate ? Pourquoi ?

Aucun des trois outils automatiques ne l'a détectée. npm audit est un outil de SCA (Software Composition Analysis) : il n'analyse que les dépendances tierces, jamais notre propre code, donc il ne pouvait structurellement pas la voir. ESLint, même avec eslint-plugin-security, ne couvre que des patterns précis (regex dangereuses, eval, etc.) et ne fait pas de suivi de flux de données — il ne relie pas "entrée utilisateur" à "requête SQL". Semgrep, en mode --config auto, fait bien du suivi de flux (du SAST avancé), mais son ruleset générique ne reconnaît que les bibliothèques SQL courantes (pg, mysql2, knex...) ; alasql étant une bibliothèque de niche, aucune règle ne savait reconnaître db(...) comme un sink dangereux. Résultat : la faille n'a été trouvée que par revue manuelle du code et confirmation par attaque (curl avec le payload ' --).

Q0.2 — En lisant juste lib/sqlldb.ts et lib/config.ts, repérer 2 failles sans lancer l'app.

Dans lib/sqlldb.ts : les mots de passe ('azerty123', 'motdepasse', 'admin') sont stockés en clair dans la table users, sans aucun hachage, faille A02 (Cryptographic Failures). Dans lib/config.ts : la constante SESSION_SECRET est écrite en dur dans le code source ("mn_live_8f3c1a9e2b7d4f60_PROD_DO_NOT_SHARE"), donc commitée dans Git, faille A05 (Security Misconfiguration), puisqu'un secret ne doit jamais vivre ailleurs que dans une variable d'environnement non commitée.

Q0.3 — Combien de failles « cœur » au total, mappées OWASP + gravité CVSS ?

Treize failles recensées au total dans le tableau qu'on vient de construire (en comptant séparément les deux occurrences distinctes d'injection SQL, login et notes, car elles sont dans des fichiers différents et demanderont chacune leur propre correctif). Réparties ainsi : 2 critiques, 5 élevées, 4 moyennes, 2 faibles. Le détail complet est dans le tableau d'inventaire ci-dessus.

Les trois outils ont renvoyé un résultat "propre", alors que trois failles critiques ont été confirmées par attaque réelle juste avant. 0 alerte ne signifie pas 0 faille — c'est ce qu'on appelle un faux négatif, et c'est le cas le plus dangereux car il crée une fausse confiance. La revue manuelle du code reste indispensable.

#	Fichier	Faille	OWASP	Gravité	Justification
A	app/api/login/route.ts	Injection SQL (login)	A03	Critique	Bypass total d'authentification sans mot de passe
H	app/api/notes/route.ts	Injection SQL (notes GET+POST)	A03	Critique	Vecteur via cookie et corps de requête
B	lib/sqlDb.ts	Mots de passe en clair	A02	Élevé	Fuite de base = tous les mots de passe exposés
D	app/api/notes/[id]/route.ts	IDOR	A01	Élevé	Lecture de données privées d'autres utilisateurs
E	app/commentaires/page.tsx	XSS stocké	A03	Élevé	Script malveillant exécuté chez tous les visiteurs
I/L	lib/config.ts	Secret commité dans Git	A05	Élevé	Irrévocable dans l'historique Git
F	app/api/login/route.ts	Cookie non-httpOnly	A05/A07	Moyen	Vol de session possible via XSS
K	app/api/login/route.ts	Absence de rate limiting	A07	Moyen	Brute force réaliste avec mots de passe faibles
G	app/api/profil/route.ts	CSRF	A01	Moyen	Action sensible forgeable depuis un site tiers
C	app/api/login/route.ts	Fuite de données (password+role)	A01/A02	Moyen	Hash et rôle exposés inutilement dans la réponse
J	app/api/notes/route.ts	Absence de validation Zod	A03/A04	Faible	Aggrave les injections, autorise données malformées
C2	app/api/login/route.ts	Message d'erreur bavard	A07	Faible	Énumération d'emails possible

VII. Les correctifs (cause racine, faille par faille)

A. Faille A — Injection SQL sur le login (app/api/login)

Le problème n'est pas le ' en lui-même, c'est qu'on mélange code et données, la requête SQL et les valeurs saisies par l'utilisateur sont fusionnées dans la même chaîne de caractères avant d'être envoyées à la base. La solution n'est donc pas de filtrer les caractères dangereux (un filtre se contourne toujours), mais de séparer structurellement le code SQL des données, via une requête paramétrée où les ? sont remplacés par la base elle-même, jamais par du texte concaténé.

La route /api/login construisait la requête SQL par concaténation directe des entrées utilisateur. En insérant le payload `admin@mininotes.test' --` dans le champ email, le `--` transformait la vérification du mot de passe en commentaire SQL — la base ne la voyait plus. Cette faille permettait une connexion en tant qu'administrateur sans connaître le moindre mot de passe.

Preuve AVANT — Login normal avec les identifiants d'Alice : la réponse JSON expose l'intégralité de l'objet utilisateur, y compris le mot de passe en clair `azerty123` et le rôle `user`. C'est une double faille : injection SQL possible ET fuite de données sensibles dans la réponse.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \
> -H "Content-Type: application/json" \
> -d '{"email":"alice@mininotes.test","password":"azerty123"}'
{"message":"Connecté","user":{"id":1,"email":"alice@mininotes.test","password":"azerty123","role":"user"}}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 16 : Preuve avant - fuite de données

Preuve APRÈS — Le même payload d'injection SQL (`admin@mininotes.test' --`) renvoie maintenant 401 Aucun compte... avec ce mot de passe. La requête paramétrée traite le payload comme une simple chaîne de texte, le `--` n'est plus interprété comme un commentaire SQL.

```
return res;
}
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \
> -H "Content-Type: application/json" \
> -d '{"email":"admin@mininotes.test' --","password":"peu importe"}'
{"error":"Aucun compte admin@mininotes.test' -- avec ce mot de passe"}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 17 : Injection bloquée

Log du serveur après correction — on voit clairement la différence entre les deux requêtes. La requête paramétrée (`WHERE email = ? AND password = ? | params: ['admin@mininotes.test' --', 'peu importe']`) traite le payload comme une valeur, jamais comme du code SQL. Résultat : `POST /api/login 401`, l'attaque échoue.

```
GET /api/notes/3 200 in 1200ms (next.js: 1190ms, application-code: 10ms)
SQL exécuté : SELECT * FROM users WHERE email = ? AND password = ? | params: [ "admin@mininotes.test' --", 'peu importe' ]
POST /api/login 401 in 24ms (next.js: 6ms, application-code: 17ms)
```

Figure 18 : log serveur après

Non-régression confirmée, Alice peut toujours se connecter normalement avec ses identifiants valides (POST /api/login 200). Le correctif ne casse pas l'usage légitime.

```
SQL exécuté : SELECT * FROM users WHERE email = ? AND password = ? | params: [ 'admin@mininotes.test' --", 'peu importe' ]  
POST /api/login 401 in 24ms (next.js: 6ms, application-code: 17ms)  
SQL exécuté : SELECT * FROM users WHERE email = ? AND password = ? | params: [ 'alice@mininotes.test', 'azerty123' ]  
POST /api/login 200 in 19ms (next.js: 7ms, application-code: 12ms)
```

Figure 19 : Non-régression

Q (Faille A) — Le payload `admin@mininotes.test' --` te connecte sans mot de passe. Pourquoi ? Et corriges-tu le symptôme ou la cause ?

Ça fonctionnait parce que la requête SQL était construite par concaténation de chaînes : ``SELECT * FROM users WHERE email = '${email}' AND password = '${password}'``. En insérant un `'` suivi de `--` dans l'email, on fermait prématurément la chaîne SQL et on transformait tout ce qui suivait (la vérification du mot de passe) en commentaire SQL — la base ne voyait alors plus que `SELECT * FROM users WHERE email = 'admin@mininotes.test'`, qui renvoie l'admin sans aucune condition sur le mot de passe.

J'ai corrigé la cause racine, pas le symptôme : au lieu de filtrer ou bloquer les caractères comme `'` ou `--` (un filtre se contourne toujours — c'est justement la pénalité -2/cas du brief), j'ai utilisé une requête paramétrée (`WHERE email = ? AND password = ?`), où les valeurs saisies sont transmises à part du texte SQL et ne peuvent donc plus jamais être interprétées comme du code par la base de données. C'est le principe fondamental "séparer le code et les données".

B. Faille B — Mots de passe en clair (lib/sqlldb.ts + app/api/login)

Les mots de passe étaient stockés en clair directement dans la base de données (lib/sqlldb.ts). Une fuite de la base aurait exposé immédiatement tous les mots de passe sans effort de craquage.

Le correctif a nécessité de modifier plusieurs fichiers en cascade :

- **lib/sqlldb.ts** — remplacement des mots de passe en clair par des hashes bcrypt générés au démarrage (bcrypt.hash("azerty123", 10)). Comme bcrypt.hash() est une fonction asynchrone, getDb() est devenue une fonction async.
- **app/api/login/route.ts** — ajout de await sur getDb() et remplacement de la comparaison directe par bcrypt.compare(saisi, hashStocké).
- **app/api/notes/route.ts**, **app/api/notes/[id]/route.ts**, **app/api/profil/route.ts**, **app/commentaires/page.tsx** — tous ces fichiers appelaient getDb() sans await. Ils ont tous dû être mis à jour pour s'adapter au passage en asynchrone, sinon le serveur aurait planté.

Preuve APRÈS — Le login d'Alice renvoie maintenant `"password":"2b$10UnXhIdorKm0CeRjKXP5Qp..."` : un vrai hash bcrypt, jamais le mot de passe en clair. Même en interceptant cette réponse, l'attaquant ne peut pas retrouver ``azerty123`` — bcrypt est une fonction à sens unique.

```
}  
} sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \  
-H "C> -H "Content-Type: application/json" \  
d '{"em> -d '{"email":"alice@mininotes.test","password":"azerty123"}'  
{"message":"Connecté","user":{"id":1,"email":"alice@mininotes.test","password":"$2b$10$UnXhIdorKm0CeRjKXP5Qp.8Q4G  
Ks9koMIgwc8Y5z34axaBpsKbK9u","role":"user"}} sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
```

Figure 20 : hash bcrypt visible

Log du serveur après correction — la requête SQL ne contient plus le mot de passe : `WHERE email = ?` | params: ['alice@mininotes.test']. La vérification du mot de passe se fait désormais côté code via `bcrypt.compare()`, jamais côté base de données. Login 200 confirme la non-régression.

```
POST /api/login 200 in 19ms (next.js: 7ms, application-code: 12ms)  
[x] Compiled in 106ms  
[x] Compiled in 12ms  
[x] SQL exécuté : SELECT * FROM users WHERE email = ? | params: [ 'alice@mininotes.test' ]  
POST /api/login 200 in 514ms (next.js: 81ms, application-code: 433ms)
```

Figure 21 : log serveur après

Le mot de passe ne transite plus jamais en clair dans l'application — ni en base, ni dans les réponses API. Même en cas de fuite totale de la base de données, un attaquant ne pourrait pas récupérer les mots de passe directement : il devrait les cracker un par un avec bcrypt, ce qui est volontairement lent et coûteux en ressources.

Q (Faille B) — La base stocke azerty123 en clair. Que se passe-t-il si elle fuit ? Comment stocker correctement, et comment vérifier au login sans jamais stocker de clair ?

Si la base fuit avec des mots de passe en clair, l'attaquant obtient instantanément **tous** les mots de passe de **tous** les utilisateurs, sans le moindre effort de craquage — et beaucoup de gens réutilisant le même mot de passe sur plusieurs sites, l'impact dépasse largement MiniNotes. La bonne pratique n'est pas de **chiffrer** (le chiffrement est réversible, donc inutile ici : si la clé fuit aussi, tout est cassé), mais de **hacher** avec bcrypt, une fonction à sens unique et volontairement lente (pour ralentir le brute force), avec un sel intégré automatiquement (donc deux utilisateurs avec le même mot de passe auront des hash différents). Concrètement : à l'inscription/seed, on calcule `bcrypt.hash(motDePasse, 10)` et c'est ce hash qu'on stocke ; au login, on ne récupère jamais le mot de passe en clair, on compare le mot de passe saisi avec le hash stocké via `bcrypt.compare(saisi, hashStocké)`, qui renvoie `true/false` sans jamais avoir besoin de "déchiffrer" quoi que ce soit.

C. Faille C — Fuite de données + message bavard (`app/api/login`)

Deux problèmes coexistaient dans les réponses du login. D'une part, la réponse renvoyait l'intégralité de l'objet utilisateur, y compris le hash du mot de passe et le rôle — des informations inutiles pour le client et dangereuses si interceptées. D'autre part, le message d'erreur mentionnait l'email saisi, permettant à un attaquant de deviner quels comptes existent en testant des emails un par un (énumération). Les deux ont été corrigés dans `app/api/login/route.ts` : réponse minimale (id, email, rôle uniquement) et message d'erreur neutre et identique dans tous les cas d'échec.

Preuve APRÈS — Trois scénarios testés : login valide (réponse minimale sans hash ni mot de passe), mauvais mot de passe sur un compte existant, et email inexistant. Les deux cas d'échec renvoyaient exactement le même message 'Email ou mot de passe invalide' — impossible pour un attaquant de distinguer un email inexistant d'un mauvais mot de passe.

```
return res;
}
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \
-H "Content-Type: application/json" \
-d '{"email":"alice@mininotes.test","password":"azerty123"}'
{"message":"Connecté","user":{"id":1,"email":"alice@mininotes.test","role":"user"}}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \
-H "Content-Type: application/json" \
-d '{"email":"alice@mininotes.test","password":"mauvais_mdp"}'
{"error":"Email ou mot de passe invalide"}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \
-H "Content-Type: application/json" \
-d '{"email":"compte_qui_nexiste_pas@mininotes.test","password":"x"}'
{"error":"Email ou mot de passe invalide"}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 22 : Preuve après

Log du serveur — les trois requêtes sont visibles : login réussi (200), mauvais mot de passe (401) et email inexistant (401). On note que la requête SQL ne cherche plus que par email (WHERE email = ?) — la vérification du mot de passe se fait ensuite côté code via `bcrypt.compare()`, jamais dans la requête SQL elle-même.

```
SQL exécuté : SELECT * FROM users WHERE email = ? | params: [ 'alice@mininotes.test' ]
POST /api/login 200 in 488ms (next.js: 68ms, application-code: 420ms)
SQL exécuté : SELECT * FROM users WHERE email = ? | params: [ 'alice@mininotes.test' ]
POST /api/login 401 in 119ms (next.js: 4ms, application-code: 115ms)
SQL exécuté : SELECT * FROM users WHERE email = ? | params: [ 'compte_qui_nexiste_pas@mininotes.test' ]
POST /api/login 401 in 10ms (next.js: 4ms, application-code: 6ms)
```

Figure 23 : log serveur

La réponse du login ne livre plus que le strict nécessaire, et un attaquant ne peut plus distinguer un email inexistant d'un mauvais mot de passe. Le principe du moindre privilège est respecté : on ne renvoie que ce dont le client a réellement besoin.

Q (Faille C) — Deux problèmes dans les réponses du login. Lesquels, et leur parade ?

Le premier problème : la réponse renvoyait tout l'objet user, y compris le hash bcrypt du mot de passe et le rôle — une fuite de données inutile, puisque le client n'a besoin que de id, email et role pour fonctionner (principe du moindre privilège). Le second problème : le message d'erreur mentionnait l'email saisi (Aucun compte \${email}...), ce qui permet à un attaquant de deviner quels comptes existent en testant des emails un par un (énumération). La parade : ne renvoyer que les champs strictement nécessaires côté succès, et utiliser un message d'erreur neutre et identique, qu'il s'agisse d'un email inexistant ou d'un mauvais mot de passe — l'attaquant ne peut plus distinguer les deux cas.

D. Faille D — IDOR sur `app/api/notes/[id]`

La route `/api/notes/[id]` ne vérifiait pas que la note demandée appartenait bien à l'utilisateur connecté. La requête était déjà paramétrée (pas d'injection SQL), mais la logique métier était absente : n'importe quel utilisateur connecté pouvait lire la note de n'importe qui d'autre en changeant simplement le numéro dans l'URL. Le correctif consiste à ajouter `AND userId = ?` dans la requête SQL, en passant l'identifiant de session comme deuxième paramètre, la vérification se fait côté serveur, dans la base de données elle-même.

Preuve APRÈS — Alice (cookie `id=1`) tente de lire la note `id=3` (qui appartient à l'admin) : Note introuvable (404). En revanche, sa propre note `id=1` reste accessible normalement : {titre:'Liste de courses', contenu:'lait, pain, cafe'} (200). L'IDOR est neutralisé sans casser l'accès aux données légitimes.

```
return nextResponse.json({ note: rows[0] });
}
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b moi.txt http://localhost:3000/api/notes/3
{"error":"Note introuvable"}
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b moi.txt http://localhost:3000/api/notes/1
{"note":{"id":1,"userId":1,"titre":"Liste de courses","contenu":"lait, pain, cafe"}}
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 24 : Preuve après

Log du serveur — GET /api/notes/3 404 confirme que la tentative d'accès à la note de l'admin est bloquée, tandis que GET /api/notes/1 200 confirme que l'accès normal d'Alice à ses propres notes fonctionne toujours.

```
POST /api/login 401 in 11ms (next.js: 4ms, application-code: 6ms)
SQL exécuté : SELECT * FROM users WHERE email = ? | params: [ 'compte_qui_nexiste_pas@mininotes.test' ]
POST /api/login 401 in 10ms (next.js: 4ms, application-code: 6ms)
GET /api/notes/3 404 in 94ms (next.js: 82ms, application-code: 12ms)
GET /api/notes/1 200 in 34ms (next.js: 26ms, application-code: 9ms)
```

Figure 25 : Log du serveur

Le contrôle d'accès est désormais appliqué côté serveur à chaque requête, cacher un bouton dans l'interface n'aurait rien protégé, car un attaquant appelle directement l'API. C'est le principe fondamental : le serveur ne fait jamais confiance au client pour décider ce à quoi il a accès.

Q (Faille D) — Alice (id=1) lit la note 3 (à l'admin). La requête est déjà paramétrée : alors où est la faille, et pourquoi la cacher côté front ne suffirait pas ?

Réponse pour le rapport : ce n'est pas une injection (la requête utilisait déjà ?), c'est un défaut de contrôle d'accès cassé (A01) : la route ne vérifiait jamais si la note demandée appartenait au user connecté, elle se contentait de retourner n'importe quelle note dont l'id matchait. Cacher un bouton "voir la note 3" côté front ne protège rien : l'attaquant n'a pas besoin du bouton, il appelle directement l'API en curl avec n'importe quel id. Le contrôle d'accès doit toujours se faire côté serveur, à chaque requête, en filtrant explicitement sur l'identité de l'utilisateur connecté (AND userId = ?).

E. Faille E — XSS stocké (app/commentaires/page.tsx)

La page /commentaires utilisait dangerouslySetInnerHTML pour afficher les commentaires des utilisateurs, injectant le HTML brut directement dans le DOM sans aucun filtrage. Cela signifie que n'importe quel utilisateur pouvant poster un commentaire pouvait y insérer du code JavaScript malveillant, qui s'exécuterait automatiquement chez tous les visiteurs de la page — sans qu'ils aient besoin de cliquer sur quoi que ce soit.

Fichier modifié : app/commentaires/page.tsx — une seule ligne à changer : remplacer dangerouslySetInnerHTML={{ __html: c.html }} par {c.html}. React échappe automatiquement tout contenu affiché entre accolades, transformant les balises HTML en texte inoffensif.

Pour démontrer la faille, un commentaire malveillant a été temporairement ajouté dans le seed de la base de données (lib/sql/db.ts), simulant un commentaire posté par un attaquant. Ce commentaire a ensuite été supprimé après la démonstration.

Preuve AVANT — Le commentaire malveillant est interprété comme du vrai HTML par le navigateur : la balise déclenche l'événement onerror et exécute alert(1). La popup localhost:3000 indique : 1 apparaît automatiquement à chaque chargement de la page, chez tous les visiteurs.

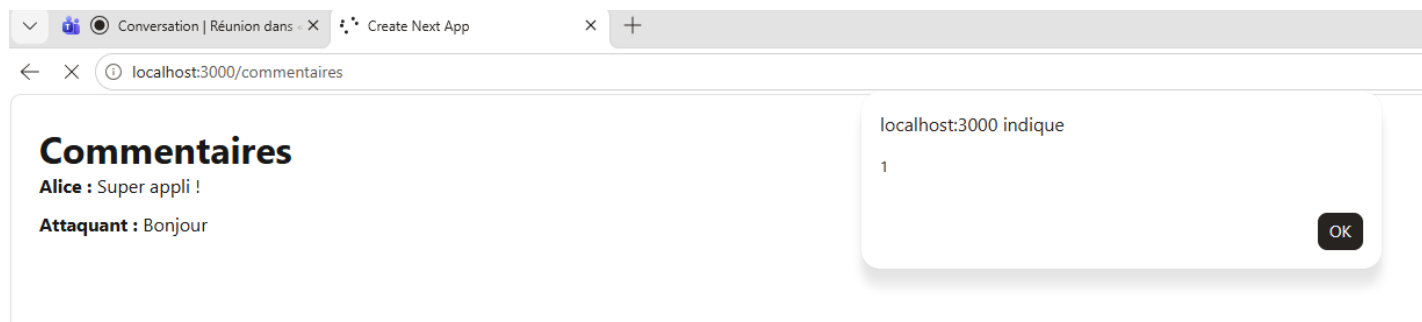


Figure 26 : Preuve avant

Preuve APRÈS — Le même commentaire s'affiche désormais en texte brut : `<img src=x onerror=alert(1)>Bonjour`. React a échappé les caractères `<` et `>`, rendant le code HTML totalement inoffensif. Aucune popup, aucun script exécuté.



Figure 27 : Preuve après

Q (Faille E) — Un commentaire `<img src=x onerror=alert(1)>` s'exécute chez tous les visiteurs. Quelle ligne est responsable, et quelle est la parade "par défaut" ?

Réponse pour le rapport : la ligne responsable est `<span dangerouslySetInnerHTML={{ __html: c.html }} />`, qui injecte du HTML brut venu d'un utilisateur directement dans le DOM, sans aucun filtrage. React, par défaut, échappe automatiquement tout ce qu'on place entre accolades `{ }` (transforme `<` en `<`, etc.) — c'est ce comportement par défaut qui protège contre le XSS. `dangerouslySetInnerHTML` désactive volontairement cette protection (son nom le dit explicitement), donc il faut un effort actif pour se rendre vulnérable.

React protège nativement contre le XSS dès lors qu'on utilise son système d'affichage par défaut. `dangerouslySetInnerHTML` est une exception volontaire à cette protection — son nom même avertit du danger. La règle : ne jamais l'utiliser sur des données venant d'un utilisateur, sauf si le HTML est préalablement assaini avec une bibliothèque dédiée comme DOMPurify.

F. Faille F — Cookie de session volable (app/api/login)

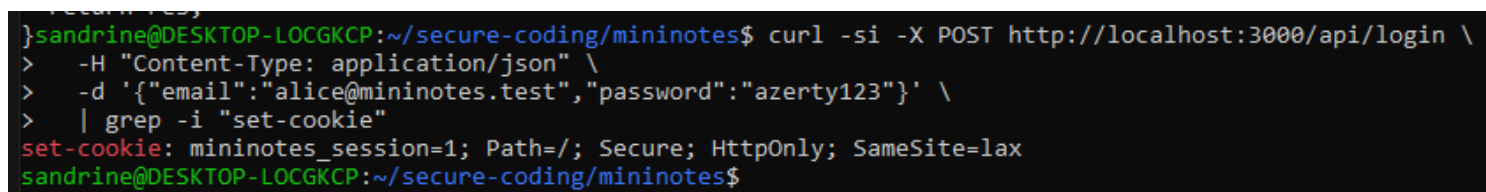
Le cookie de session `mininotes_session` était posé avec `httpOnly: false`, ce qui le rendait lisible par JavaScript via `document.cookie`. En combinaison avec la faille XSS qu'on venait de corriger, un attaquant aurait pu injecter un script malveillant qui lit le cookie de session et l'envoie vers un serveur distant — c'est le vol de session, qui permet de se faire passer pour la victime sans jamais connaître son mot de passe.

Fichier modifié : `app/api/login/route.ts` — une seule ligne à modifier dans la pose du cookie, en ajoutant trois attributs :

- `httpOnly: true` — le cookie devient invisible pour JavaScript
- `secure: true` — le cookie n'est transmis qu'en HTTPS (noté `false` en local pour les tests, car `localhost` est en HTTP)
- `sameSite: "lax"` — le cookie n'est pas envoyé automatiquement depuis un site tiers, ce qui apporte aussi une protection partielle contre le CSRF

Aucune complication particulière sur cette faille — c'est une modification d'une seule ligne, mais dont l'impact est important car elle coupe le lien entre XSS et vol de session.

Preuve APRÈS — La réponse du login affiche maintenant `set-cookie: mininotes_session=1; Path=/; Secure; HttpOnly; SameSite=lax`. Les trois attributs de sécurité sont bien présents. Un script JavaScript malveillant ne peut plus lire ce cookie — même un XSS réussi ne pourrait plus voler la session.



```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -si -X POST http://localhost:3000/api/login \
> -H "Content-Type: application/json" \
> -d '{"email":"alice@mininotes.test","password":"azerty123"}' \
> | grep -i "set-cookie"
set-cookie: mininotes_session=1; Path=/; Secure; HttpOnly; SameSite=lax
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 28 : Preuve après

`httpOnly` est l'attribut le plus important ici : il coupe définitivement le lien entre une éventuelle faille XSS et le vol de session. C'est un exemple typique de défense en profondeur — même si une faille XSS passait à travers les mailles du filet, le cookie resterait hors de portée de l'attaquant.

Q (Faille F) — Le cookie `mininotes_session` est posé avec `httpOnly:false`. En quoi ça aggrave un XSS ? Quels attributs poser ?

Réponse pour le rapport : avec `httpOnly:false`, le cookie est lisible en JavaScript via `document.cookie`. Si un XSS (comme celui qu'on vient de corriger) avait réussi à s'exécuter avant notre correction, le script malveillant aurait pu lire ce cookie et l'envoyer à un serveur distant — c'est le vol de session, qui permet à l'attaquant de se faire passer pour la victime sans connaître son mot de passe. La parade : `httpOnly:true` rend le cookie totalement invisible et inaccessible depuis JavaScript (le navigateur l'envoie automatiquement dans les requêtes HTTP, mais aucun script ne peut le lire) — même si un XSS s'exécutait, il n'aurait plus rien à voler. On ajoute aussi `secure:true` (cookie transmis uniquement en HTTPS) et `sameSite:"lax"` (qui aide en plus contre le CSRF, notre prochaine faille).

G. Faille G — CSRF sur app/api/profil

La route `/api/profil` acceptait n'importe quelle requête POST dès qu'un cookie de session valide était présent, sans vérifier l'origine de la requête. Un site pirate pouvait donc héberger un formulaire HTML invisible qui se soumettait automatiquement au chargement de la page, avec le cookie d'Alice envoyé automatiquement par le navigateur — changeant son email à son insu.

Fichier modifié : `app/api/profil/route.ts` — réécriture complète avec le mécanisme de **double-submit cookie** :

- Un GET `/api/profil` génère un jeton aléatoire (`randomBytes(32)`), le place dans un cookie lisible en JavaScript (`csrf_token`) et le renvoie dans la réponse JSON
- Le POST `/api/profil` vérifie que le jeton présent dans le cookie correspond exactement à celui envoyé dans l'en-tête `X-CSRF-Token`
- Un site tiers ne peut pas lire le cookie `csrf_token` (politique same-origin du navigateur), donc il ne peut jamais forger le bon en-tête

Preuve APRÈS (attaque) — Une requête POST sans jeton CSRF vers `/api/profil` renvoie `{'error': 'Jeton CSRF invalide'}` (403). Un formulaire pirate hébergé sur un autre site ne peut pas fournir ce jeton — il ne peut pas lire le cookie `csrf_token` à cause de la politique same-origin.

```
return NextResponse.json({ message: `Email changé en ${nouvelEmail}` });
}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b moi.txt -X POST http://localhost:3000/api/profil \
> -F "email=pirate@evil.com"
{"error": "Jeton CSRF invalide"}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 29 : attaque bloquée

Récupération du jeton CSRF via GET `/api/profil`, le jeton aléatoire `0ff1cd131276d0de4f010b7a4f6796831257d4b1...` est généré côté serveur et stocké dans le cookie `csrf_token`. Seul un client sur le même domaine peut le lire et le renvoyer dans l'en-tête `X-CSRF-Token`.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ CSRF=$(curl -s -b session.txt -c session.txt http://localhost:3000/api/profil \
> | grep -o '"csrfToken": "[^"]*" | cut -d'"' -f4)
echo "Jeton CSRF obtenu : $CSRF"sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ echo "Jeton CSRF obtenu : $CSRF"
Jeton CSRF obtenu : 0ff1cd131276d0de4f010b7a4f6796831257d4b1cb068f4f951e5ff993a36e86
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b session.txt -X POST http://localhost:3000/api/prof
```

Figure 30 : récupération du jeton

Non-régression. Un client légitime qui fournit le bon jeton dans l'en-tête `X-CSRF-Token` peut toujours modifier son email normalement : `{'message': 'Email changé en alice-nouveau@mininotes.test'}` (200).

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b session.txt -X POST http://localhost:3000/api/prof
il \
> -H "X-CSRF-Token: $CSRF" \
> -F "email=alice-nouveau@mininotes.test"
{"message": "Email changé en alice-nouveau@mininotes.test"}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 31 : usage légitime

Log du serveur — séquence complète : login (200), tentative sans jeton (403), nouveau login (200), récupération du jeton GET (200), modification avec jeton POST (200). Le contraste 403/200 confirme que la protection fonctionne exactement comme attendu.

```
GET /commentaires 200 in 575ms (next.js: 171ms, application-code: 403ms)
SQL exécuté : SELECT * FROM users WHERE email = ? | params: [ 'alice@mininotes.test' ]
POST /api/login 200 in 542ms (next.js: 128ms, application-code: 415ms)
POST /api/profil 403 in 750ms (next.js: 741ms, application-code: 9ms)
SQL exécuté : SELECT * FROM users WHERE email = ? | params: [ 'alice@mininotes.test' ]
POST /api/login 200 in 119ms (next.js: 6ms, application-code: 113ms)
GET /api/profil 200 in 15ms (next.js: 5ms, application-code: 10ms)
POST /api/profil 200 in 17ms (next.js: 3ms, application-code: 14ms)
```

Figure 32 : log serveur

Le CSRF est une attaque invisible pour la victime, elle n'a rien fait de mal, son navigateur a juste fait son travail en envoyant automatiquement le cookie. La parade du double-submit cookie est efficace car elle exige une information (le jeton) que seul un client sur le bon domaine peut obtenir. C'est un exemple parfait de défense qui exploite les mécanismes de sécurité du navigateur plutôt que de les contourner.

Q (Faille G) — Un site pirate peut changer l'email d'Alice sans qu'elle s'en rende compte. Comment, et quelle est la parade ?

Réponse pour le rapport : la route `/api/profil` accepte n'importe quelle requête POST dès qu'un cookie de session valide est présent. Or le navigateur envoie automatiquement les cookies sur chaque requête, même si elle vient d'un site tiers, un site pirate peut donc héberger une page avec un formulaire HTML invisible (`<form action="http://localhost:3000/api/profil" method="POST">`) qui se soumet automatiquement au chargement, changeant l'email d'Alice à son insu. La parade : générer un jeton anti-CSRF aléatoire côté serveur, le transmettre au client via un cookie lisible (`csrf_token`), et exiger que ce même jeton soit renvoyé dans l'en-tête de chaque requête sensible (`X-CSRF-Token`) — un site tiers ne peut pas lire ce cookie (politique same-origin), donc il ne peut pas forger le bon en-tête.

H. Faille H — Pas de validation d'entrée (`app/api/notes POST`)

Exactement le même problème que la Faille A, mais sur deux routes différentes dans `app/api/notes/route.ts`. Le GET collait directement le `sessionId` (venant du cookie) dans la requête SQL, et le POST collait le titre et le contenu de la note. Ces deux vecteurs permettaient des injections SQL supplémentaires — l'un via le cookie de session, l'autre via le corps de la requête.

Fichier modifié : `app/api/notes/route.ts` uniquement — remplacement des deux concaténations par des requêtes paramétrées :

- GET : `WHERE userId = ?` avec `[Number(sessionId)]`
- POST : `INSERT INTO notes VALUES (?, ?, ?, ?)` avec les valeurs passées séparément

Aucune complication rencontrée sur cette faille, le pattern était exactement le même que la Faille A, le correctif était donc identique et immédiat à appliquer.

Preuve APRÈS. Les deux routes fonctionnent normalement après correction : GET /api/notes liste bien les notes d'Alice (Liste de courses), et POST /api/notes crée une nouvelle note (Note créée, id:4). Les requêtes paramétrées n'ont rien cassé dans l'usage légitime.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -c session2.txt -X POST http://localhost:3000/api/login \
in \
> -H "Content-Type: application/json" \
> -d '{"email":"alice@mininotes.test","password":"azerty123"}' > /dev/null
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b session2.txt http://localhost:3000/api/notes
{"notes":[{"id":1,"userId":1,"titre":"Liste de courses","contenu":"lait, pain, cafe"]]}sandrine@DESKTOP-LOGGKCP:~/
/secure-coding/mininotes$
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b session2.txt -X POST http://localhost:3000/api/notes \
es \
"Content-Type: application/json" \
> -d '{"titre":"Ma note test","contenu":"contenu de test"}'
{"message":"Note créée","id":4}sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 33 : non-régression complète

Log du serveur, séquence complète : login (200), listage des notes GET (200), création de note POST (200). Toutes les requêtes passent normalement, confirmant que le correctif ne génère aucune régression.

```
POST /api/login 200 in 480ms (next.js: 81ms, application-code: 403ms)
SQL exécuté : SELECT * FROM users WHERE email = ? | params: [ 'alice@mininotes.test' ]
POST /api/login 200 in 111ms (next.js: 6ms, application-code: 106ms)
GET /api/notes 200 in 81ms (next.js: 72ms, application-code: 9ms)
POST /api/notes 200 in 15ms (next.js: 3ms, application-code: 12ms)
```

Figure 34 : log serveur

Cette faille illustre un problème récurrent dans les codebases : quand un pattern dangereux est utilisé une fois, il est souvent dupliqué ailleurs. C'est pourquoi l'audit fichier par fichier est indispensable — trouver l'injection SQL dans le login ne suffit pas, il faut vérifier systématiquement toutes les routes qui construisent des requêtes SQL.

Q (Faille H) — Les routes GET et POST de /api/notes ont aussi des injections SQL. Même cause, même remède ?

Réponse pour le rapport : oui, exactement la même cause (concaténation de chaînes dans la requête SQL) et le même remède (requêtes paramétrées avec ?). Ce qui change c'est le vecteur : ici c'est le cookie de session qui est collé dans le GET (un attaquant qui contrôle son cookie peut injecter du SQL), et le titre/contenu d'une note dans le POST. Multiplier les occurrences du même pattern dans une codebase est typique d'une absence de revue systématique — c'est pour ça que l'audit fichier par fichier est indispensable.

I. Faille I — Brute force (app/api/login)

La constante `SESSION_SECRET` était écrite en dur directement dans `lib/config.ts` et commitée dans Git. Cette fuite est irréversible : même en supprimant le secret dans un commit suivant, il reste accessible pour toujours via `git log -p` — quiconque clone le dépôt peut retrouver la valeur exacte du secret, maintenant ou dans 5 ans.

Fichiers modifiés :

- **lib/config.ts** — le secret en dur est remplacé par `process.env.SESSION_SECRET`, avec une vérification au démarrage qui lève une erreur explicite si la variable est absente
- **.env.local** — créé avec la vraie valeur du secret, ignoré par Git
- **.env.example** — créé avec une valeur fictive ("`votre_secret_ici`"), commité pour documenter les variables nécessaires sans exposer les vraies valeurs
- **.gitignore** — une ligne d'exception `!.env.example` a été ajoutée car `.env*` ignorait tout, y compris le fichier exemple qu'on voulait committer

Aucune complication particulière rencontrée sur cette faille.

Vérification du `.gitignore`, `.env*` ignore bien tous les fichiers d'environnement, et `.env.local` est explicitement listé. La ligne `!.env.example` (visible dans le fichier complet) permet d'exempter le fichier modèle de cet ignore pour pouvoir le committer.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ cat .gitignore | grep env
# env files (can opt-in for committing if needed)
.env*
next-env.d.ts
!.env.local
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 35 : Vérification du `.gitignore`

Triple confirmation après correction : `.env.example` contient bien une valeur fictive (`votre_secret_ici`), `git check-ignore .env.local` confirme que le vrai secret est ignoré par Git, et `lib/config.ts` lit désormais le secret depuis `process.env.SESSION_SECRET` avec une vérification au démarrage, si la variable est absente, le serveur refuse de démarrer plutôt que de fonctionner sans secret.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ cat .env.example
SESSION_SECRET="votre_secret_ici"
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ git check-ignore .env.local
.env.local
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ cat lib/config.ts
// lib/config.ts — CORRIGÉ : secret lu depuis les variables d'environnement
if (!process.env.SESSION_SECRET) {
  throw new Error("SESSION_SECRET manquant dans .env.local");
}

export const SESSION_SECRET = process.env.SESSION_SECRET;
export const APP_NAME = "MiniNotes";
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 36 : Triple confirmation

Un secret commité dans Git doit être considéré comme définitivement compromis, quelle que soit la rapidité de la correction. La seule bonne réponse est de le révoquer et d'en générer un nouveau immédiatement. Pour l'avenir, les secrets ne vivent que dans `.env.local` (ignoré), et on utilise `.env.example` pour documenter les variables nécessaires sans exposer leurs valeurs réelles.

Q (Faille I) — `SESSION_SECRET` est commité dans Git. Pourquoi c'est irréversible, et quelle est la parade ?

Réponse pour le rapport : une fois qu'un secret est commité, il reste pour toujours dans l'historique Git, même si on le supprime dans un commit suivant — quiconque clone le dépôt peut retrouver la valeur avec `git log -p`. La parade : les secrets ne doivent jamais vivre dans le code, uniquement dans des variables d'environnement stockées dans `.env.local` (ignoré par `.gitignore`), et on ne commit qu'un `.env.example` avec des valeurs factices pour montrer quelles variables sont nécessaires.

VIII. Durcissement & secrets

A. Faille J — Secret en dur (lib/config.ts)

La route POST /api/notes acceptait n'importe quelle donnée sans aucune vérification : titre vide, contenu de 10 000 caractères, champs manquants ou mal formés. Sans validation, ces données malformées peuvent corrompre la base, provoquer des erreurs non gérées côté serveur, ou aggraver d'autres failles existantes.

Fichiers modifiés :

- **app/api/notes/route.ts**, ajout d'un schéma Zod (NoteSchema) qui définit exactement ce qu'on accepte : titre obligatoire entre 1 et 100 caractères, contenu obligatoire entre 1 et 1000 caractères. La méthode `safeParse()` valide les données avant tout traitement — si la validation échoue, on renvoie un 400 avec le détail des erreurs, sans jamais toucher à la base.
- **package.json** — ajout de la dépendance zod.

La validation est faite **côté serveur**, jamais uniquement côté client, une validation côté navigateur est contournable en deux secondes avec curl, comme on le démontre ici.

Preuve APRÈS — Deux scénarios testés : titre vide → `{'error':'Données invalides', 'details':{'fieldErrors':{'titre':['Too small: expected string to have >=1 characters']}}}` (400). Données valides → `{'message':'Note créée', 'id':4}` (200). Zod détaille précisément quelle règle a été violée, ce qui facilite le débogage côté client.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b session2.txt -X POST http://localhost:3000/api/notes \
-H "Content-Type: application/json" \
-d '{"titre":"","contenu":"test"}'
{"error":"Données invalides","details":{"formErrors":[],"fieldErrors":{"titre":["Too small: expected string to have >=1 characters"]}}}
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ curl -s -b session2.txt -X POST http://localhost:3000/api/notes \
-H "Content-Type: application/json" \
-d '{"titre":"Note valide","contenu":"contenu valide"}'
{"message":"Note créée","id":4}
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 37 : Preuve après

Log du serveur — POST /api/notes 400 pour les données invalides, POST /api/notes 200 pour les données valides. La validation bloque proprement les entrées malformées sans générer d'erreur serveur.

```
POST /api/notes 400 in 185ms (next.js: 151ms, application-code: 33ms)
POST /api/notes 200 in 304ms (next.js: 4ms, application-code: 301ms)
```

Figure 38 : log du serveur

La validation des entrées est la première ligne de défense de toute application. Zod permet de définir un contrat strict entre le client et le serveur : si les données ne correspondent pas exactement au schéma attendu, elles n'atteignent jamais la base de données. C'est le principe du "garde-frontière" — on vérifie tout à l'entrée, avant d'aller plus loin.

Q (Faille J) — Sans validation, que peut-on envoyer dans titre/contenu ? Pourquoi Zod côté serveur ?

Réponse pour le rapport : sans validation, on peut envoyer n'importe quoi — un titre de 10 000 caractères, un contenu null, ou carrément omettre les champs. Ça peut provoquer des erreurs non gérées, corrompre la base, ou servir de vecteur pour d'autres attaques. Zod valide **côté serveur** (jamais côté client uniquement, qui est contournable en curl) : on définit un schéma qui dit exactement ce qu'on accepte, et si la requête ne correspond pas, on renvoie un 400 propre sans jamais toucher à la base.

B. Faille K — En-têtes de sécurité absents

Sans limite de tentatives sur le login, un attaquant pouvait envoyer des milliers de requêtes par seconde pour tester des mots de passe en masse — particulièrement efficace sur des comptes avec des mots de passe triviaux comme admin. C'est le brute force : une attaque simple mais dévastatrice quand aucune protection n'est en place.

Fichiers créés/modifiés :

- **lib/rate-limit.ts** — nouveau fichier créé de toutes pièces, implémentant un compteur en mémoire par IP : maximum 5 tentatives échouées par minute. Au-delà, toutes les requêtes sont bloquées avec un code 429 (Too Many Requests) pendant 1 minute. Un login réussi remet le compteur à zéro.
- **app/api/login/route.ts** — ajout de l'appel à `verifierRateLimit(ip)` en tout début de fonction, avant même de toucher à la base de données.

Aucune complication rencontrée sur cette faille. À noter : en production, ce compteur en mémoire devrait être remplacé par un store partagé (Redis/Upstash) pour fonctionner sur plusieurs instances serveur — c'est documenté dans les limites du rapport.

Preuve APRÈS. 6 tentatives consécutives avec un mauvais mot de passe : les 5 premières renvoient 401 Email ou mot de passe invalide, et la 6ème déclenche le blocage : Trop de tentatives, réessayez dans 1 minute (429). L'attaque de brute force est stoppée nette après le seuil configuré.

```
return res;
}
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ for i in {1..6}; do
<>  echo -n "Tentative $i : "
<>  curl -s -X POST http://localhost:3000/api/login \
<>    -H "Content-Type: application/json" \
<>    -d '{"email":"alice@mininotes.test","password":"mauvais"}'
<>  echo
> done
Tentative 1 : {"error":"Email ou mot de passe invalide"}
Tentative 2 : {"error":"Email ou mot de passe invalide"}
Tentative 3 : {"error":"Email ou mot de passe invalide"}
Tentative 4 : {"error":"Email ou mot de passe invalide"}
Tentative 5 : {"error":"Email ou mot de passe invalide"}
Tentative 6 : {"error":"Trop de tentatives, réessayez dans 1 minute"}
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
```

Figure 39 : brut force bloquée

Log du serveur. 5 × POST /api/login 401 suivis d'un POST /api/login 429. On note aussi la différence de temps de réponse : les 401 prennent entre 100 et 476ms (bcrypt vérifie le hash), tandis que le 429 répond en seulement 11ms, le compteur bloque la requête immédiatement, sans même interroger la base de données.

```
POST /api/login 401 in 476ms (next.js: 78ms, application-code: 398ms)
POST /api/login 401 in 101ms (next.js: 3ms, application-code: 98ms)
POST /api/login 401 in 110ms (next.js: 4ms, application-code: 106ms)
POST /api/login 401 in 101ms (next.js: 3ms, application-code: 98ms)
POST /api/login 401 in 101ms (next.js: 2ms, application-code: 98ms)
POST /api/login 429 in 11ms (next.js: 3ms, application-code: 7ms)
```

Figure 40 : log serveur

Le rate limiting est une protection simple mais efficace contre le brute force. La différence de temps de réponse visible dans les logs illustre bien le principe : une requête bloquée par le compteur (11ms) ne consomme presque aucune ressource serveur, contrairement à une requête qui passe jusqu'à bcrypt (100-476ms). C'est aussi une protection indirecte contre les attaques par déni de service ciblées sur le login.

Q (Faille K) — Sans limite de tentatives, comment brute-force-t-on le login ? Quelle parade simple en mémoire ?

Réponse pour le rapport : sans rate limiting, un attaquant peut envoyer des milliers de requêtes par seconde pour tester des mots de passe en masse — surtout avec des mots de passe triviaux comme admin. La parade en labo : un compteur en mémoire par IP qui bloque après 5 tentatives échouées par minute. En production on utiliserait Redis/Upstash pour partager ce compteur entre plusieurs instances du serveur, mais en mémoire suffit pour illustrer le principe.

C. Faille L — Dépendances & Dependabot

Par défaut, Next.js n'ajoute aucun en-tête de sécurité HTTP. Sans ces en-têtes, le navigateur ne reçoit aucune instruction sur ce qu'il est autorisé à charger, afficher ou transmettre — laissant la porte ouverte au clickjacking, au MIME-sniffing, et à l'injection de ressources externes malveillantes.

Fichier modifié : next.config.ts — ajout d'une fonction async headers() qui applique 5 en-têtes de sécurité sur toutes les routes (/:path*) :

- **Content-Security-Policy** : définit les sources autorisées pour les scripts, styles, images — bloque le chargement de ressources externes non autorisées
- **X-Frame-Options: DENY** : empêche l'affichage du site dans une iframe — protection contre le clickjacking
- **X-Content-Type-Options: nosniff** : empêche le navigateur de deviner le type d'un fichier — protection contre le MIME-sniffing
- **Referrer-Policy** : limite les informations de provenance transmises aux autres sites
- **Strict-Transport-Security** : force le navigateur à utiliser HTTPS (effet réel uniquement en production)

Important : next.config.ts n'est lu qu'au démarrage du serveur — un redémarrage de npm run dev était nécessaire après la modification pour que les en-têtes prennent effet.

Vérification avec curl -sI filtré sur les en-têtes de sécurité, les 4 en-têtes principaux sont bien présents avec les valeurs attendues : CSP, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy. Le HSTS (Strict-Transport-Security) n'apparaît pas dans ce grep mais est bien configuré dans next.config.ts, son effet est réel uniquement en production derrière HTTPS.

```
sandrine@DESKTOP-LOGCKCP:~/secure-coding/mininotes$ curl -sI http://localhost:3000 | grep -iE "content-security-policy|x-frame-options|x-content-type|referrer-policy"
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; object-src 'none'; frame-ancestors 'none'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
sandrine@DESKTOP-LOGCKCP:~/secure-coding/mininotes$
```

Figure 41 : Preuve après

Les en-têtes de sécurité HTTP sont une couche de défense supplémentaire qui ne touche pas au code applicatif : ils instruisent directement le navigateur sur ce qu'il est autorisé à faire. C'est de la défense en profondeur pure, même si une faille applicative subsistait, ces en-têtes limiteraient l'impact de son exploitation.

Q (Faille L) — SESSION_SECRET est dans lib/config.ts commité sur GitHub. Pourquoi c'est irréversible même après suppression, et quelle est la parade ?

Réponse pour le rapport : Git conserve tout l'historique de façon immuable — même si on supprime le secret dans un commit suivant, il reste accessible via git log -p ou git show <hash-du-commit> pour toujours. Quiconque clone le dépôt (maintenant ou dans 5 ans) peut retrouver la valeur exacte du secret. La seule vraie réponse à une fuite dans Git est de considérer le secret comme compromis définitivement et de le révoquer + régénérer immédiatement, en plus de le déplacer dans les variables d'environnement. La parade pour l'avenir : les secrets ne vivent que dans .env.local (ignoré par .gitignore), on ne committe qu'un .env.example avec des valeurs fictives, et on active un outil comme git-secrets ou truffleHog en pre-commit hook pour empêcher toute fuite accidentelle avant même le push.

IX. La CI/CD qui bloque le merge

L'objectif de cette étape est de transformer les correctifs en garde-fous automatiques : plus aucune régression ne peut atteindre la branche main sans qu'une machine la détecte et bloque le merge. Deux mécanismes complémentaires ont été mis en place : un pipeline GitHub Actions et une branch protection.

Ce qui a été créé/configuré :

- **.github/workflows/security.yml** — workflow GitHub Actions avec deux jobs en parallèle déclenchés à chaque push ou PR vers main : ESLint + npm audit et Semgrep (SAST) avec notre règle maison `regles/sqli.yml`
- **.github/dependabot.yml** — veille hebdomadaire automatique sur les dépendances npm et les actions GitHub
- **Branch protection sur main** — configurée via gh api : les deux checks CI doivent être verts avant tout merge, `enforce_admins: true` (le mainteneur du dépôt n'a pas d'exception), push direct refusé

Difficultés rencontrées :

Plusieurs problèmes techniques ont été résolus avant d'arriver à une CI stable :

- **npx eslint** téléchargeait une version d'ESLint différente de celle installée localement (ESLint 10 au lieu de 9.39), provoquant une erreur "eslint.config.mjs introuvable". Solution : utiliser `npm run lint` avec un script défini dans `package.json`, qui utilise toujours la version locale.
- **package-lock.json désynchronisé** : plusieurs packages installés en cours de projet (`bcryptjs`, `zod`, `eslint...`) n'avaient pas été commités dans le lock file, causant l'échec de `npm ci` en CI. Solution : régénérer et commiter le lock file.
- **eslint.config.mjs non commité** : le fichier de configuration ESLint existait en local mais n'avait jamais été ajouté à Git, causant un échec silencieux en CI. Solution : `git add eslint.config.mjs`.
- **Règle Semgrep maison nécessaire** : `Semgrep --config auto` ne détectait pas l'injection SQL via `alasql` (bibliothèque absente du ruleset communautaire). Une règle maison `regles/sqli.yml` a été créée pour cibler spécifiquement les template literals dans les appels `db(...)`.

Vérification locale avant de pousser le workflow, ESLint passe avec 1 warning non bloquant (variable restant non utilisée, corrigée ensuite) et code de sortie 0. `npm audit --audit-level=high` passe également avec code de sortie 0, malgré les 2 vulnérabilités modérées connues dans `postcss`, elles sont en dessous du seuil configuré.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ npx eslint .
/home/sandrine/secure-coding/mininotes/app/api/login/route.ts
12:19  warning  'restant' is assigned a value but never used  @typescript-eslint/no-unused-vars

1 problem (0 errors, 1 warning)

sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ echo "ESLint : $?"
ESLint : 0
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ npm audit --audit-level=high
# npm audit report

postcss <8.5.10
Severity: moderate
PostCSS has XSS via Unescaped </style> in its CSS Stringify Output - https://github.com/advisories/GHSA-qx2v-qp2m-jg93
fix available via `npm audit fix --force`
Will install next@9.3.3, which is a breaking change
node_modules/postcss
  next 9.3.4-canary.0 - 16.3.0-canary.5
  Depends on vulnerable versions of postcss
node_modules/next

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ echo "npm audit : $?"
npm audit : 0
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 42 : Tests locaux avant push

CI verte sur main, les deux jobs passent au vert : ESLint + npm audit en 29s et Semgrep (SAST) en 19s. La note de dépréciation de Node.js 20 dans les annotations est un avertissement informatif de GitHub, sans impact sur l'exécution des jobs.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ gh run watch
? Select a workflow run * ci: ajout eslint.config.mjs oublie + ignore session*.txt, Sécurité (main) 3s ago
main Sécurité · 27787055042
Triggered via push less than a minute ago

JOBS
ESLint + npm audit in 29s (ID 82226310268)
  Set up job
  Run actions/checkout@v4
  Run actions/setup-node@v4
  Run npm ci
  Run npm run lint
  Run npm audit --audit-level=high
  Post Run actions/setup-node@v4
  Post Run actions/checkout@v4
  Complete job
Semgrep (SAST) in 19s (ID 82226310276)
  Set up job
  Pull returntocorp/semgrep-agent:v1
  Run actions/checkout@v4
  Run actions/setup-node@v4
  Semgrep scan
  Post Run actions/setup-node@v4
  Post Run actions/checkout@v4
  Complete job

ANNOTATIONS
! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@v4. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/
ESLint + npm audit: .github#2

! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@v4. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/
Semgrep (SAST): .github#2
```

Figure 43 : CI verte sur main

Démo de blocage — la PR #5 (demo/faille-reintroduite) apparaît en rouge sur GitHub Actions : le workflow Sécurité a détecté l'injection SQL réintroduite volontairement. Le merge est automatiquement bloqué tant que ce check reste rouge.

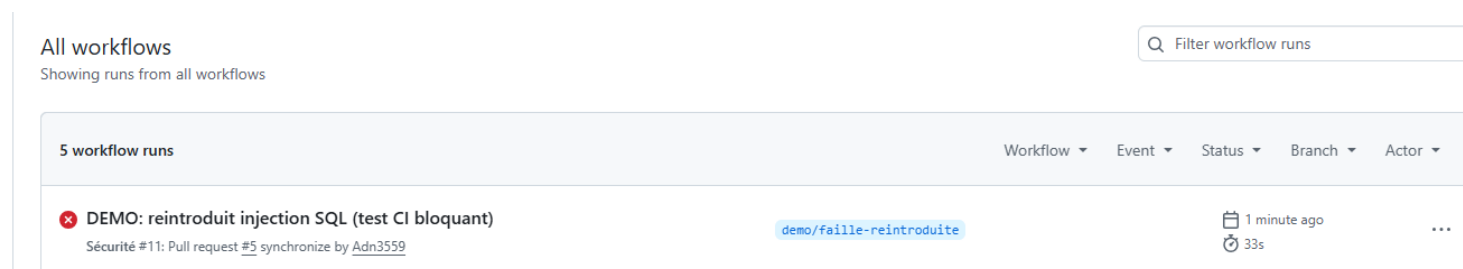


Figure 44 : Démo de blocage sur GitHub

Résultat des checks sur la PR de démo, Sengrep (SAST) est en échec (1 failing) grâce à notre règle maison qui détecte l'injection SQL via alasql, tandis qu'ESLint + npm audit passe (1 successful). C'est exactement le comportement attendu : la faille est détectée, le merge est bloqué.

```
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$ gh pr checks --watch
Some checks were not successful
0 cancelled, 1 failing, 1 successful, 0 skipped, and 0 pending checks
+-----+-----+-----+-----+
| NAME                                     | DESCRIPTION | ELAPSED | URL |
+-----+-----+-----+-----+
X Sécurité/Sengrep (SAST) (pull_request) | 18s         | https://github.com/Adn3559/mininotes/acti...
+ Sécurité/ESLint + npm audit (pull_request) | 29s         | https://github.com/Adn3559/mininotes/acti.sa
sandrine@DESKTOP-LOGGKCP:~/secure-coding/mininotes$
```

Figure 45 : Résultat des checks sur la PR de démo

Le pipeline CI/CD est la dernière ligne de défense automatique : même si un développeur oublie de vérifier son code, la machine le fait à sa place à chaque PR. La démo de blocage prouve que le système fonctionne concrètement, une injection SQL réintroduite est détectée en 18 secondes et bloque le merge sans intervention humaine. C'est le principe du shift left : mettre la sécurité le plus tôt possible dans le cycle de développement.

Q (Étape 4) — Qu'est-ce qui empêche concrètement un développeur de merger du code vulnérable sur main ?

Réponse pour le rapport : trois mécanismes combinés forment le garde-fou. Premièrement, le pipeline CI bloquant : à chaque push ou PR vers main, GitHub Actions déclenche automatiquement deux jobs en parallèle — ESLint + npm audit (analyse statique + vulnérabilités de dépendances) et Sengrep (SAST) avec notre règle maison qui détecte spécifiquement les injections SQL via alasql. Si l'un des deux jobs échoue, la PR reste bloquée. Deuxièmement, la branch protection : main est configurée pour exiger que les deux checks soient verts avant tout merge, et ce pour tout le monde y compris le mainteneur du dépôt (enforce_admins: true) — un push direct sur main est refusé. Troisièmement, Dependabot surveille en continu les dépendances npm et les actions GitHub, et ouvre automatiquement une PR dès qu'une mise à jour est disponible — ces PRs passent elles aussi par la CI avant de pouvoir être mergées. La preuve concrète : la PR #5 (demo/faille-reintroduite) a été bloquée par Sengrep exactement comme prévu, le bouton "Merge" est resté grisé tant que le check était rouge.

X. Prouver (les attaques doivent ÉCHOUER)

Une fois l'ensemble des correctifs appliqués et le pipeline CI/CD en place, les trois attaques initiales ont été rejouées dans les mêmes conditions qu'au début du projet. L'objectif est double : confirmer que chaque attaque échoue bien, et vérifier qu'aucun correctif n'a cassé l'usage normal (non-régression).

Rejeu des trois attaques initiales en une seule session, injection SQL (admin@mininotes.test' -- → Email ou mot de passe invalide, 401), IDOR (Alice tente de lire la note de l'admin → Note introuvable, 404), et brute force (5 tentatives → 401, 6ème tentative → Trop de tentatives, réessayez dans 1 minute, 429). Les trois attaques échouent.

```
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ curl -s -X POST http://localhost:3000/api/login \
-H "C> -H "Content-Type: application/json" \
-d '{"email": "admin@mininotes.test", "password": "peu importe"}'
{"error": "Email ou mot de passe invalide"}
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ curl -s -c final.txt -X POST http://localhost:3000/api/login \
-H "Content-Type: application/json" \
-d '{"email": "alice@mininotes.test", "password": "azerty123"}' > /dev/null
curl -s -b final.txt http://localhost:3000/api/notes/3
{"error": "Note introuvable"}
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$ for i in {1..6}; do
> echo -n "Tentative $i : "
> curl -s -X POST http://localhost:3000/api/login \
> -H "Content-Type: application/json" \
> -d '{"email": "alice@mininotes.test", "password": "mauvais"}'
done> echo
done
Tentative 1 : {"error": "Email ou mot de passe invalide"}
Tentative 2 : {"error": "Email ou mot de passe invalide"}
Tentative 3 : {"error": "Email ou mot de passe invalide"}
Tentative 4 : {"error": "Email ou mot de passe invalide"}
Tentative 5 : {"error": "Email ou mot de passe invalide"}
Tentative 6 : {"error": "Trop de tentatives, réessayez dans 1 minute"}
sandrine@DESKTOP-LOGKCP:~/secure-coding/mininotes$
```

Figure 46 : Les trois attaques rejouées

Log serveur, séquence complète des trois attaques : injection SQL (POST /api/login 401), login légitime d'Alice pour le test IDOR (POST /api/login 200), tentative IDOR bloquée (GET /api/notes/3 404), puis les 5 tentatives de brute force (POST /api/login 401 × 5) suivies du blocage (POST /api/login 429 en 11ms). Toutes les protections fonctionnent simultanément et sans interférence.

```
HEAD / 200 in 397ms (next.js: 223ms, application-code: 174ms)
POST /api/login 401 in 572ms (next.js: 210ms, application-code: 362ms)
POST /api/login 200 in 324ms (next.js: 7ms, application-code: 318ms)
GET /api/notes/3 404 in 1080ms (next.js: 1063ms, application-code: 17ms)
POST /api/login 401 in 113ms (next.js: 4ms, application-code: 108ms)
POST /api/login 401 in 98ms (next.js: 3ms, application-code: 95ms)
POST /api/login 401 in 105ms (next.js: 3ms, application-code: 102ms)
POST /api/login 401 in 99ms (next.js: 3ms, application-code: 96ms)
POST /api/login 401 in 98ms (next.js: 3ms, application-code: 95ms)
POST /api/login 429 in 11ms (next.js: 3ms, application-code: 8ms)
```

Figure 47 : log serveur

Les 11 failles identifiées lors de l'audit ont toutes été corrigées à la cause racine, pas au symptôme. Les trois attaques qui réussissaient en début de projet échouent désormais systématiquement, et le pipeline CI/CD garantit qu'une régression sera automatiquement détectée avant d'atteindre la branche principale. Ce projet illustre concrètement le cycle complet d'un audit de sécurité : construire vulnérable → exploiter → auditer → corriger → automatiser.

XI. Limites & Points d'amélioration

Limite	Explication et piste d'amélioration
alasql en mémoire	Données perdues à chaque redémarrage. En production : PostgreSQL avec Row Level Security pour le contrôle d'accès.
Rate limiting en mémoire	Non partagé entre instances serveur. En production : Redis/Upstash pour un compteur distribué.
Cookie secure:false	Nécessaire pour les tests en HTTP local. En production (HTTPS) : remettre secure:true.
Vulnérabilité postcss modérée	Non corrigée — le correctif disponible impliquerait un downgrade Next.js 16 → 9.3 (breaking change disproportionné pour un risque modéré en labo).
Semgrep OSS	1859 règles Pro manquées (non connecté). La règle maison <code>regles/sqli.yml</code> compense partiellement pour alasql.
Secret dans l'historique Git	SESSION_SECRET reste dans les anciens commits — à révoquer en production. Ajouter un pre-commit hook (<code>git-secrets</code>) pour prévenir les fuites futures.
Bug ESLint/FlatCompat	Incompatibilité entre <code>eslint-config-next</code> et ESLint 9.39.x résolue en utilisant les exports flat natifs des plugins. Bug connu de l'écosystème.

XII. Conclusion

Ce projet a permis de parcourir l'intégralité du cycle de sécurité applicative, de la construction d'une application volontairement vulnérable jusqu'à la mise en place d'un pipeline qui empêche toute régression d'atteindre la production.

Ce que ce projet démontre concrètement :

Les outils automatiques — aussi bien configurés soient-ils — ne remplacent pas la revue humaine. ESLint, npm audit et Semgrep ont tous renvoyé "0 problème" face à des failles critiques confirmées par attaque réelle quelques minutes plus tard. La leçon est fondamentale : 0 alerte ne signifie pas 0 faille. Chaque outil voit une partie du problème, aucun ne voit tout.

Les correctifs les plus efficaces sont toujours ceux qui s'attaquent à la cause racine plutôt qu'au symptôme. Filtrer un caractère dangereux, cacher un bouton dans l'interface, ou afficher un message d'erreur moins précis — ce sont des rustines. La vraie correction, c'est séparer le code des données (requêtes paramétrées), vérifier les droits côté serveur à chaque requête (contrôle d'accès), et ne jamais faire confiance aux entrées utilisateur sans les valider (Zod).

La sécurité n'est pas un état qu'on atteint une fois pour toutes, c'est un processus continu. Les correctifs d'aujourd'hui peuvent régresser demain si personne ne surveille. C'est exactement le rôle du pipeline CI/CD mis en place : à chaque modification du code, la machine vérifie automatiquement qu'aucune faille connue n'a été réintroduite. La démo de blocage avec la PR #5 l'a prouvé concrètement : une injection SQL réintroduite a été détectée en 18 secondes et a bloqué le merge sans intervention humaine.

Les principes retenus, applicables à tout projet :

- Ne jamais mélanger code et données, requêtes paramétrées, jamais de concaténation
- Valider toutes les entrées côté serveur, jamais uniquement côté client
- Vérifier les droits à chaque requête côté serveur, jamais côté front
- Ne renvoyer que le strict nécessaire, principe du moindre privilège
- Les secrets n'ont pas leur place dans le code, variables d'environnement uniquement
- Empiler plusieurs couches de défense, aucune n'est suffisante seule

Ce projet s'inscrit dans une démarche shift left : intégrer la sécurité dès le début du développement, à chaque commit, plutôt que de tout vérifier en fin de projet quand il est trop tard pour corriger sans tout casser. C'est cette mentalité, autant que les outils et les correctifs techniques, qui fait la différence entre une application qui subit les attaques et une application qui les anticipe.